Answer the following questions:

A. Describe Dijkstra and A* algorithms, what are the difference between the two?

- A* and Dijkstra are two pathfinding algorithms that order nodes based on priorities to determine an optimal path between two points. These priorities are based on the distance to the goal, usually calculated by magnitude.
- The primary difference between A* and Dijkstra is that A* makes use of a heuristic, which basically means that on top of the distance to reach the node, an added "heuristic weight" is considered where this "heuristic weight" is the distance from that node to the goal, compared to Dijkstra which just calculates the distance to the goal for every node until the goal is the best choice to make.
- What this means in terms of difference in pathfinding, is that A* will often calculate fewer potential paths before it reaches the goal.

B. Which game genres are you likely to use A* with and why?

- A* is used best in games that have preset maps and have objects that traverse the world. Games like Tower Defence games, and Open World Games are great for this.
- A* is better in these cases, as it is quite expensive to search all possible paths to the goal and distance from the goal is a reliable "hint" towards which to search first.

C. Which game genres would you NOT use A* with and why?

- We wouldn't use A* in games where distance is an unreliable heuristic, for example games including mazes, since obstacles could create a much larger true path to the goal than distance from the goal may suggest.

D. What limitations do path finding algorithms put on game design and development.

- Pathfinding algorithms can be computationally expensive, so the more complex and algorithm is, the less likely you are going to be able to run it without hitting hardware constraints. For example, if a game relies on having many AI's perform pathfinding at once, then developers need to find algorithms that can be computed cheaply, or algorithms where efficiency gains can be achieved through other means like batch or pre-calculating.
- Pathfinding algorithms must also be taken into consideration in the game design, as some methods of pathfinding require a specific data structure to operate on. For example, Dijkstra's algorithm requires a graph to generate a path for traversal.

E. List 4 algorithms and/or paradigms do AI use in games.

- Dijkstra's Algorithm (For Pathfinding)
- Finite State Machines (For Simple Behavior States)
- Monty Carlo Tree Search (For Decision Making)
- Utility AI (For Decision Making)

Reflection:

I believe that the project went well, even though some Wishlist items could not be met. While I wish I could have added more features to the game, I understand that this assignment is about how pathfinding algorithms affect game development and how they are implemented.

Adding complex algorithms takes developer time and requires a code architecture to be accommodating for those algorithms which can increase code complexity.

Pathfinding algorithms can be computationally expensive, especially if using an algorithm like Dijkstra, which calculates all paths until finding a suitable one. For example, if I was making a wave with many enemies, each one uses its own pathfinder, meaning it must compute the entire graph of nodes. If this project was commercial, it would be placing constraints on the map size and the wave size, as these two things are computationally coupled - increasing one would exponentially affect the other. This forces developers to spend time trying to optimize this relationship so that to meet other requirements, for example larger maps.

Similarly, the use of pathfinding algorithms is enough to completely change the technical feasibility of a game or project. For example, in this assignment I had to restrict the ability to place towers while enemies were calculating their paths. This is because implementing response changes to the enemy paths would require them to constantly recalculate the node graph, which is severely inefficient. As mentioned previously, to make this technically feasible, I would have to change the underlying implementation of the pathfinding logic so that it could be computed in batches for example. While this is feasible for a small project, it may be impossible to do so in a larger project due to cost and time associated with the technical complexity of this change.

Pathfinding algorithms are not always suited to the user's needs in certain types of games. For example, in a simple tower defence game it would probably be enough to send enemies down a hard coded track rather than to use an entire pathfinding algorithm. A very popular game, Bloons Tower Defence, does not make use of pathfinding algorithms and rather just sends enemies down an unchanging path, and remains as a very successful game. This implies that this simple behaviour is enough to satisfy players in this genre. On the other hand, some games do need complex pathfinding innately, such as RTS' or Open World RPGs, else otherwise the AIs will feel unsatisfactory to the players.